



SZ: composable error-bounded lossy compression for scientific datasets & handson

Sheng Di

Mathematics and Computer Science Division

Argonne National Laboratory

Kai Zhao, Xin Liang, Jiannan Tian, Jinyang Liu, Zizhe Jian, Yafan Huang, Jiajun Huang, Shihui Song, Robert Underwood, Franck Cappello

Outline

- SZ's Uniqueness
- SZ's Compression Objectives and Usage
- SZ's Design Considerations
- SZ's Compression Pipeline
- SZ's Approximation
- SZ's Coding
- Performance Evaluation
- GPU Versions of SZ
- Handson: Installation, quick start, test, supporting python, etc.
- Summary

SZ's Uniqueness: More than a software

SZ is a modular parametrizable lossy compressor framework.
Clarriciation of different generic-purpose SZ-series compressors:

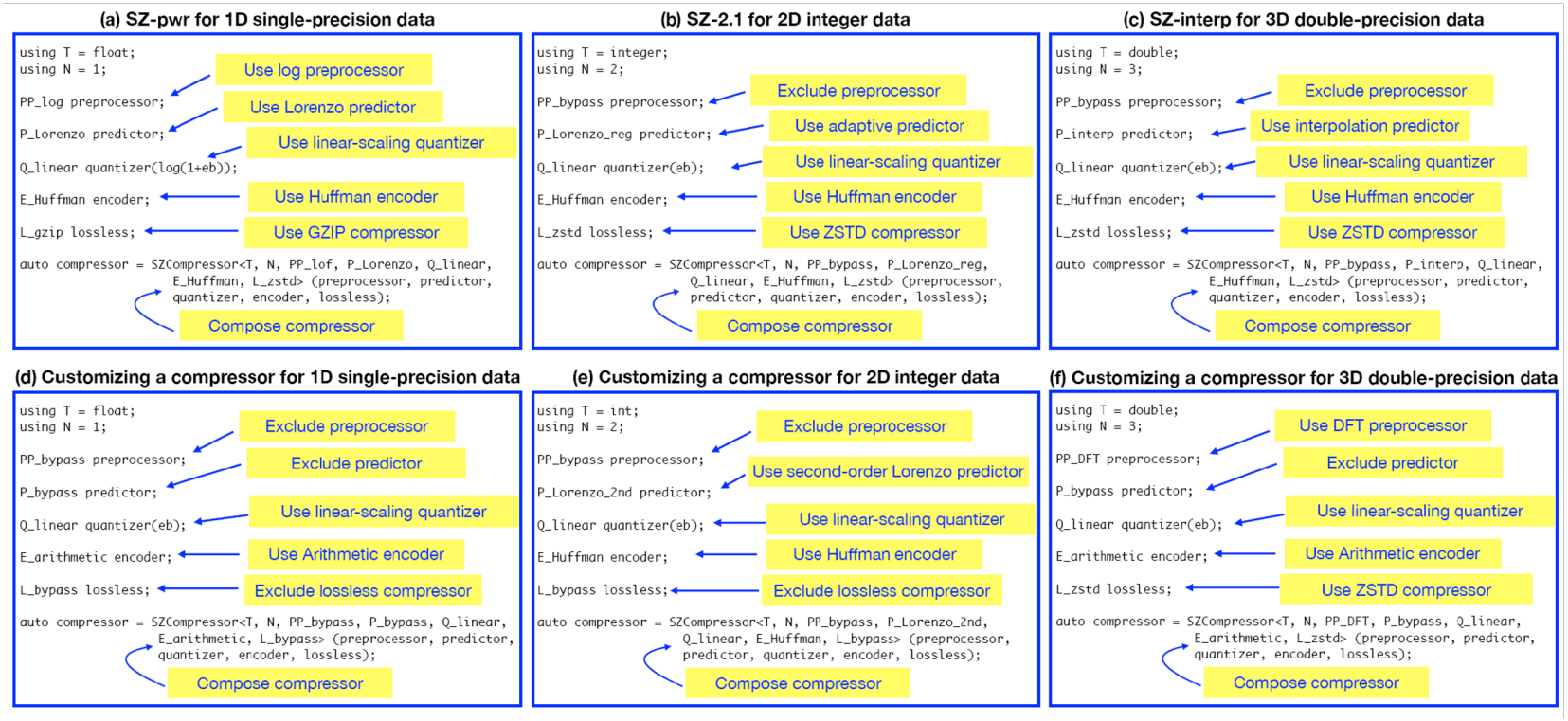
	Compression Ratio	Compression Speed
1. SZx & SZp:	Low	Best
2. SZ2:	Acceptable	Excellent
3. SZ3:	Good	Good
4. QoZ:	Excellent	Good
5. FAZ:	Best	Low

We integrated most of SZ versions into one package: SZ3.

Core-group(Lead)
Argonne National Laboratory
Dr. Franck Cappello,
Dr. Sheng Di
Dr. Robert Underwood
Florida State University
Dr. Kai Zhao
University of Kentucky
Dr. Xin Liang
University of Houston
Dr. Jinyang Liu
Temple University
Dr. Sian Jin
Oakland University
Dr. Jiannan Tian
University of Iowa
Yafan Huang
Clemson University
Dr. Jon Calhoun
University of South California
Dr. Jiajun Huang
Other Contributors
Dr. Dingwen Tao, Dr. Martin Herbordt, Dr. QingqingXiong, Dr. Wen Xia, Dr. XiangyuZou,

SZ's Uniqueness: More than a software

SZ3 is a composable framework that can customize different compression pipelines based on user-requirement or datasets.



SZ's Uniqueness: More than a software

- **SZ's Uniqueness: More than a software (2021-2024)**

We created multiple effective compression pipelines for different applications/use-cases.

Interp-SZ (default setting in SZ3): "Optimized for seismic imaging (oil and gas)". Use: improve ratio — IEEE ICDE2021
> Predictor: Multi-dimensional spline interpolation based prediction

Pastri-SZ: "Optimized for quantum chemistry". Use: avoid recomp — IEEE CLUSTER2018 (best paper)
> Predictor: Scaled-pattern based prediction

MMD-SZ (MDZ): "Optimized for material molecular dynamics". Use: reduce storage footprint — IEEE ICDE2022
> Predictor: Multi-peak distribution based prediction in space + time-based prediction

Roibin-SZ: "Optimized for X-ray crystallography (instrument)". Use: reduce storage footprint and I/O cost — Journal: SNR 2023
> Pipeline: Pixel-binning + SZ2.1 + preserving region-of-interests

Quality-oriented SZ (QoZ): Error-bounding + tuning user-specified quality metric. Use: improve quality — IEEE/ACM SC2022
> Predictor: quality-oriented predictor

Constraint-supported SZ: "Error-bounding + meeting dynamic constraints". Use: improve quality — IEEE TPDS 2022
> Pipeline: Preprocessing + traditional prediction + improved quantization bins

Feature-preserving based SZ: Toward Feature-Preserving Vector Field Compression. Use: quality — IEEE TVCG 2022
> Pipeline: Multi-dimensional spline interpolation + adjusted quantization bins.

Ocelet: "Optimizing Scientific Data Transfer on Globus with Error-bounded Lossy Compression". Use: Reduce network communication cost on WAN — IEEE ICDCS 2023
> Usecase: FuncX + SZ3 + globus

CliZ: "Optimizing compression ratio and quality for Climate datasets". Use: Improve CR for Climate — IEEE IPDPS 2024
> Predictor: Optimizing prediction based on four properties of climate data

HPEZ/QoZ 2.0: "significantly improved compression ratio". Use: Improve CR/quality — ACM SIGMOD 2024
> Predictor: interpolation re-ordering, multi-dimensional interpolation, and natural cubic splines

SZ's Uniqueness: More than a software

- ## SZ's Uniqueness: More than a software (2025)

We created multiple effective compression pipelines for different applications/use-cases.

unstructure-SZ: "Efficient Error-controlled Lossy Compression for Unstructured Data". Use: improve ratio for unstructured datasets — IEEE IPDPS2025 (best paper)
> Predictor: customized predictor for unstructured datasets

LCP: "Enhancing Lossy Compression for Particles". Use: improve compression ratio — ACM SIGMOD2025
> Predictor: customized predictor for particle datsaets

ghZCCL: "Advancing GPU-aware Collective Communications with Compression". Use: improve communication — ACM ICS2025
> Usecase: Improve MPI collectives by lossy compression

QPET: "A Versatile and Portable QoI-preservation Framework for Error-Bounded Lossy Compression". Use: Preserve qu... — VLDB2025
> Pipeline: Preserving quality for lossy compression.

lsComp: "lsCOMP: Efficient Light Source Compression". Use: Reduce storage and communication cost — IEEE/ACM SC2025
> Pipeline: Customized compression pipeline for light-source datasts

cuSZp3: "GPU Lossy Compression for HPC Can Be Versatile and Ultra-Fast". Use: Improve compression p... — IEEE/ACM SC2025
> Pipeline: Customized compression pipeline for GPU

FedEFsz: "FedEFsz: Fair Cross-Silo Federated Learning System with Error-Bounded Lossy Compression". Use: F... communication — IEEE TPDS, 2025
> Usecase: Customized compressor and strategy for federated learning

SZ's Uniqueness: More than a software

- **SZ's Uniqueness: More than a software (2026)**

We created multiple effective compression pipelines for different applications/use-cases.

pMSz: "Correcting Morse-Smale Segmentations for Lossy Compression". Use: Improv IEEE IPDPS2026
> Pipeline: adding a correction layer after the compression pipeline

PackKV: "Reducing KV Cache Memory Footprint through LLM-Aware Compression". Use: Improve int IEEE IPDPS2026
> Predictor: customized predictor for KV Cache

FFCz: "Fast Fourier Correction for Spectrum-Preserving Lossy Compression". Use: Improve c IEEE IPDPS2026
> Pipeline: adding a correction layter

EmbdC: "Error-Bounded Lossy Video Embedding Compression". Use: Reduce storage and communicat ACM CCGrid2026
> Pipeline: customized strategy for vedio embedding datsets

DeepEBC: "Compressing the Pre-Trained LLMs with Lossy Compression". Use: Reduce communication BDXCS2026
> Usecase: study how to improve pre-trained LLM data compression

OPAL: "On-demand Progressive Accelerated Scientific Lossy Compression. Use: Reduce communica ACM HPDC2026
> Pipeline: enabling progressive compression for lossy compression

Fixed-rate Framework: "From Error-Bounded to Fixed-Ratio". Use: Im IEEE/ACM SC2026
> Pipeline: customize fixed-ratio compression

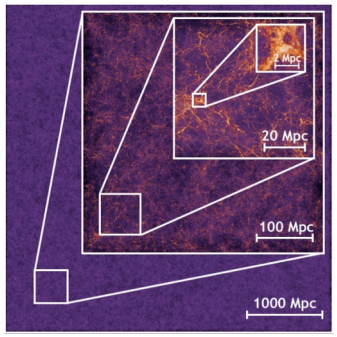
hZCCL2: "Co-Designing Collective Communication with Vectorized Homomorphic Compression". Use: Reduce communication cost IEEE/ACM SC2026
> Pipeline and usecase: Customize more efficient lossy compression for collectives

SZ's Compression Objectives and Usage

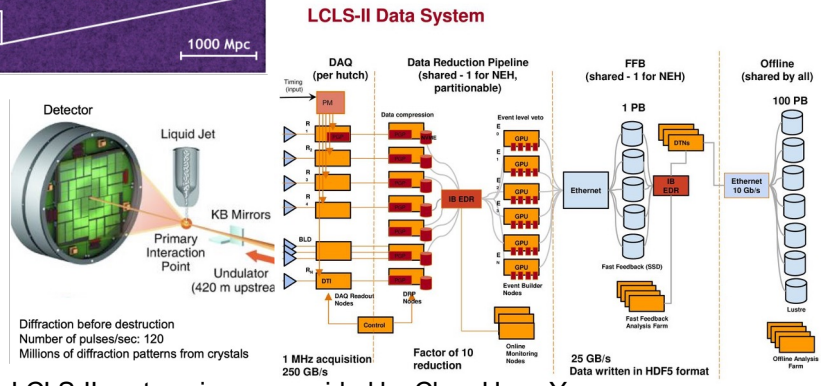
- **SZ is designed for significantly reducing data storage and transfer burden caused by extremely large volumes of data produced by today's scientific simulations**
- **Cosmology simulation:**
  - A total of **>20PB** of data when simulating 1 trillion of particles (500 snapshots)
  - Petascale systems never gave 20PB for one project
  - On current file system (1TB/s), storing the 20PB may take 20X10^15 seconds (**5h30**).
- **LCLS-II:**
  - 4M pixel/event
  - All LCLS-II data detectors per experiment: **250 GB/s**
  - Today technology: ~x1000s disks to sustain the bandwidth
    - Objective of compression ratio: **10:1**



LCLS-II Image downloaded from https://lcls.slac.stanford.edu/lcls-ii



Zoom-in visualization of the density field in a 1.07 trillion particle, 4.225 Gpc box-size simulation with HACC on 32 Blue Gene/Q racks. (Figure extracted from Habib et al. "HACC: Simulating Sky Surveys on State-of-the-Art Supercomputing Architectures", New Astronomy, 2016.



LCLS-II system: image provided by Chun Hong Yoon.
Source: https://slideplayer.com/slide/12645601/

SZ's Compression Objectives and Usage

Application domains supported by SZ:

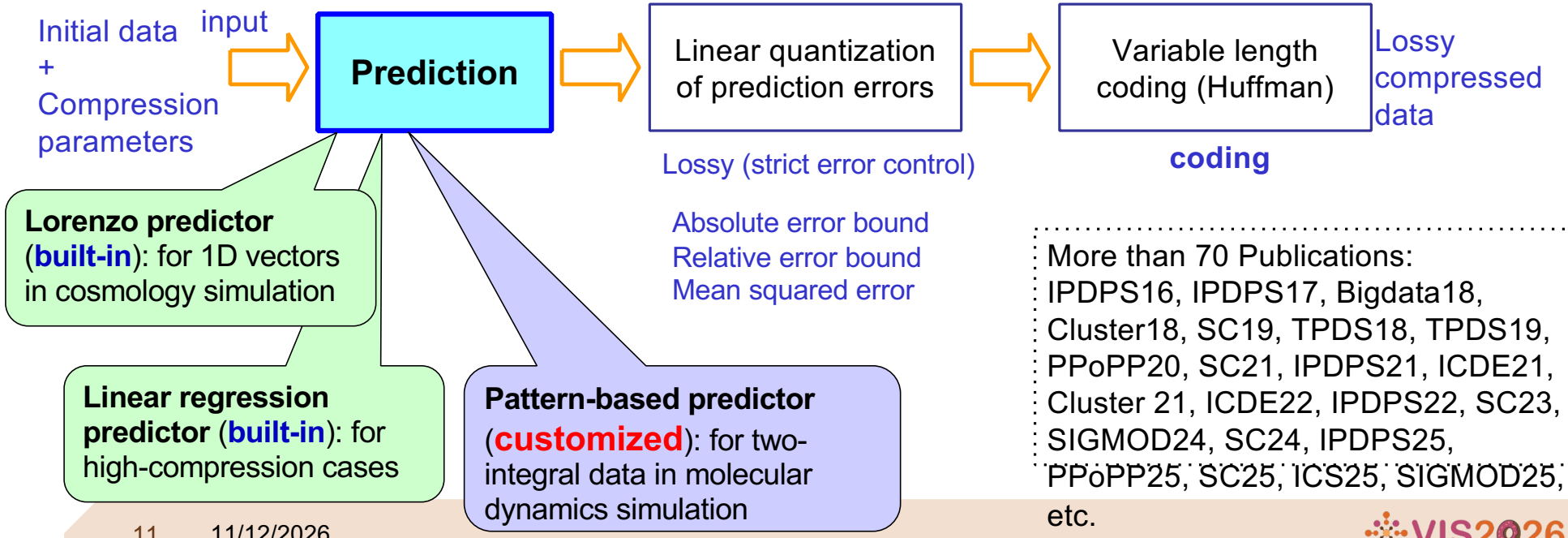
- Fluid dynamics: climate/weather (ECP)
- Particle physics: cosmology, molecular dynamics (ECP)
- Fusion energy: plasma simulation (ECP)
- Seismology (RTM): Oil and Gas
- Quantum chemistry (ECP)
- Quantum Computing simulation
- Medical imaging
- Physics instruments: light sources (LCLS, APS) (ECP)
- Deep learning models
- Accelerate MPI Collective Communications

SZ Design Considerations (Key features)

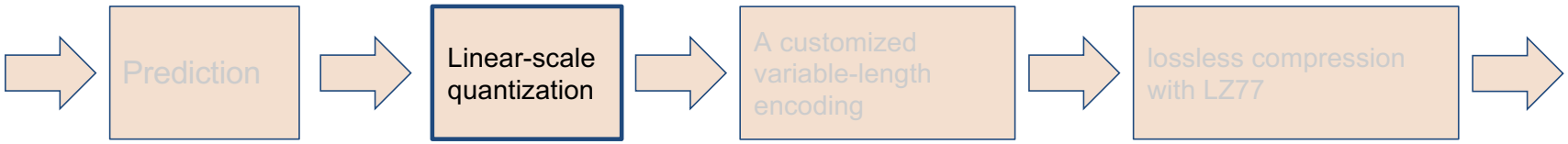
- **Strict error controls (absolute error, relative error, PSNR)**
- **Support both structured and unstructured datasets:** E.g. 3D simulation fields, 2D instruments data, time series.
- **Support Various Data Types**: For **floating point** and **integer** data
- **Support Multiple Languages**: C, C++, Python, etc.
- **Support Multiple I/O Reader/Writer**: ADIOS, HDF5, netCDF
- **Optimized compression ratios** (support multiple predictors: linear-regression, Lorenzo, time-based, etc.)
- **High compression/decompression speed** (*MPI, OpenMP, cuDA, Cerebras, FPGA*)

SZ's Compression Pipeline

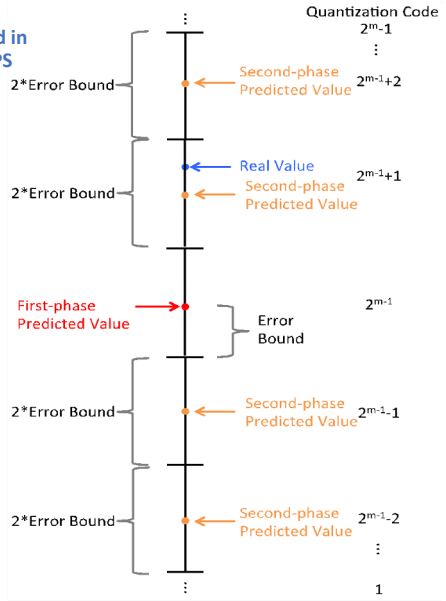
- Goal: Produce a high-quality (**production quality**) lossy compressor responding to the demands of various scientific applications.
- SZ: a fast error-controlled lossy compressor for scientific datasets
  - SZ adopts a **flexible prediction-based** compression framework



SZ's Approximation: Quantization

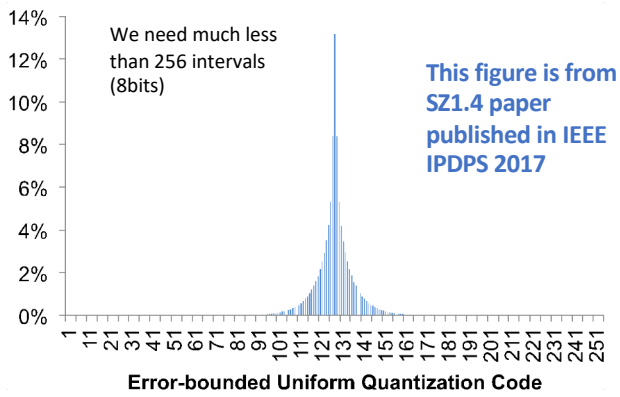


This figure is from SZ1.4 paper published in IEEE IPDPS 2017



- lossy strict error bound

Example for err: 10^{-4}, for ATM



This figure is from SZ1.4 paper published in IEEE IPDPS 2017

Example: hit rate for ATM

This figure is from SZ1.4 paper published in IEEE IPDPS 2017

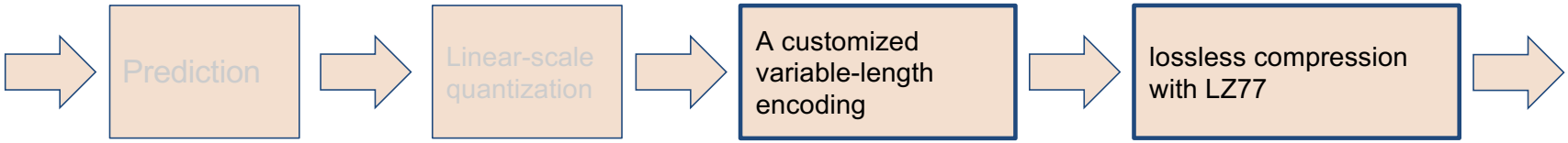
- Transforms each predicted data into 1 integer value (with loss)
- If data is out of scale, keep it in a separate array

D. Tao, S. Di, Z. Chen, F. Cappello, Significantly Improving Lossy Compression for Scientific Datasets Based on Multidimensional Prediction and Error-Controlled Quantization, **IEEE IPDPS 2017**

SZ's Coding

Classic coding method on **CPU**: Huffman encoding + Zstd
Tailored coding method for **GPU**:

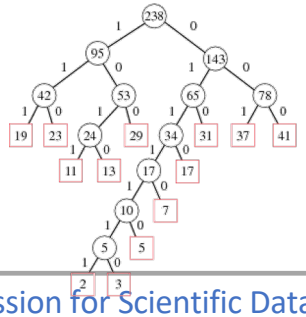
- GPU version of Huffman encoding + GPU version of LZ77 (published in IPDPS22, ICS23)
- constant-block-folding + blockwise fixed-length encoding (published HPDC22, ICS23, SC23)



- Lorenzo (1-layer non-linear prediction)
- extended Lorenzo (2-layer prediction)
- Linear-regression based prediction
- mean value prediction
- pattern-aware prediction
- ……

- lossy strict error bound

- Huffman encoding
- Arithmetic encoding



- Zstd
- Zlib

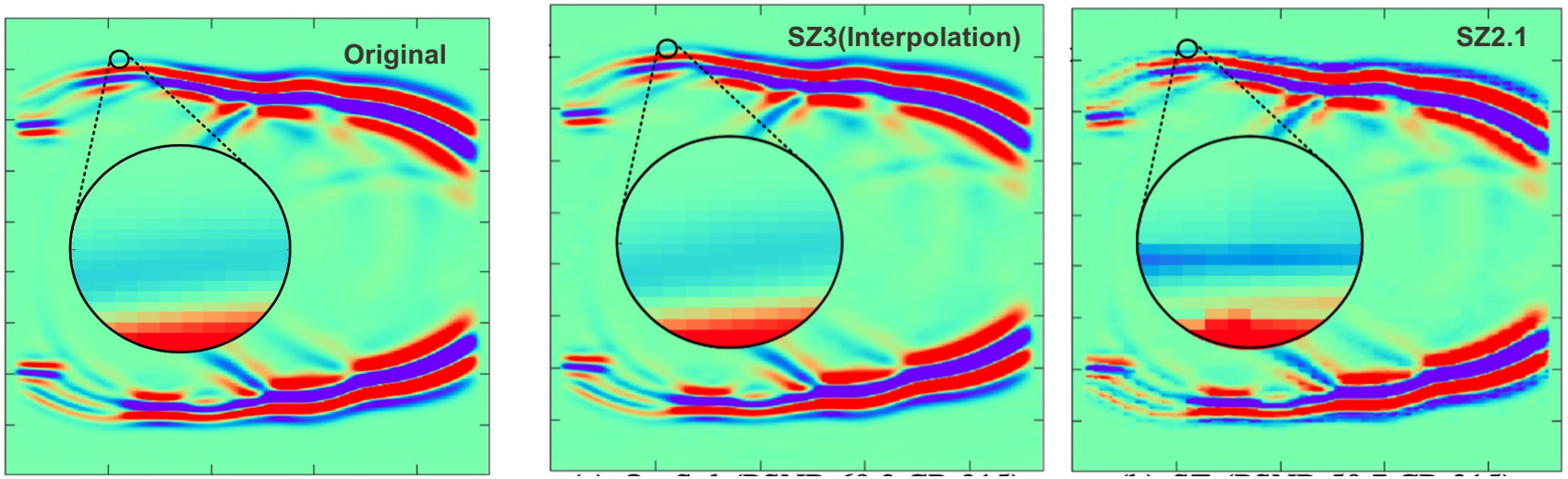
Optional dictionary encoding: (L77 + Finite State Entropy): **Zstandard** by default. (Zstandard is much faster than GZIP: improves speed of SZ 2.0 compared to SZ 1.4)

D. Tao, S. Di, Z. Chen, F. Cappello, Significantly Improving Lossy Compression for Scientific Datasets Based on Multidimensional Prediction and Error-Controlled Quantization, **IEEE IPDPS 2017**

Performance Evaluation (SZ3 vs. SZ2)

Visual Quality

3D Compression Ratio: ~300:1 for difference compressors



Compression Ratio = 315:1

This figure is from our paper published in IEEE ICDE2021

Kai Zhao,Sheng Di, Maxim Dmitriev, Thierry-Laurent D. Tonellot, Zizhong Chen, and Franck Cappello, "Optimizing Error-Bounded Lossy Compression for Scientific Data by Dynamic Spline Interpolation", Proceeding of the 37th IEEE International Conference on Data Engineering (ICDE2021), Chania, Crete, Greece, Apr 19 - 22, 2021.

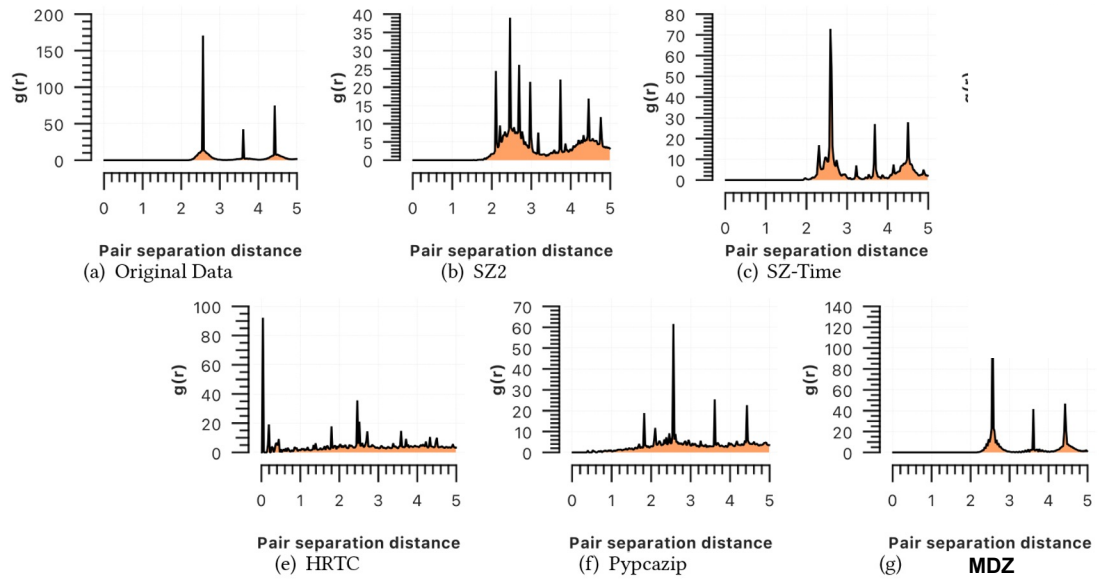
Performance Evaluation (MMD-SZ (MDZ))

Related Work: HRTC [JCC2016]: adopts piecewise linear representation of trajectories

Pycazip [SoftwareX16]: PCA-based compression for molecular dynamics data

New prediction method:
> Multi-level patterns in space
> High-smoothness in time dimension

Source: Kai Zhao, Sheng Di, Danny Perez, Xin Liang, Zizhong Chen, Franck Cappello, "MDZ: An Efficient Error-bounded Lossy Compressor for Molecular Dynamics", in Proceeding of the 38th IEEE International Conference on Data Engineering (ICDE2022), Virtual Event, May 9-12, 2022



Radial distribution function (RDF) of original and decompressed data (Copper-B, CR=10)

GPU versions based on SZ framework

1. cuSZ: https://github.com/szcompressor/cuSZ (high compression ratio)
PACT2020, IPDPS2021, CLUSTER2021, IPDPS2022, ICS2023, etc.
2. cuSZx: https://github.com/szcompressor/cuSZx (high kernel speed)
Xiaodong Yu, Sheng Di, Kai Zhao, Jiannan Tian, Dingwen Tao, Xin Liang, Franck Cappello, "Ultra-fast Error-bounded Lossy Compression for Scientific Dataset", ACM HPDC2022.
3. FZ-GPU: https://github.com/szcompressor/FZ-GPU (high E2E speed)
Boyuan Zhang, Jiannan Tian, Sheng Di, Xiaodong Yu, Yunhe Feng, Yunhe Feng, Xin Liang, Dingwen Tao, Franck Cappello, "FZ-GPU: A Fast and High-Ratio Lossy Compressor for Scientific Computing Applications on GPUs," ACM HPDC2023.
4. cuSZp: https://github.com/szcompressor/cuSZp (high E2E speed)
Yafan Huang, Sheng Di, Xiaodong Yu, Guanpeng Li, Franck Cappello, "cuSZp: An Ultra-fast GPU Error-bounded Lossy Compression Framework with Optimized End-to-End Performance", IEEE/ACM SC2023.
5. cuSZp2: https://github.com/szcompressor/cuSZp (further improved speed)
Yafan Huang, Sheng Di, Guanpeng Li, Franck Cappello, "cuSZp2: A GPU Lossy Compressor with Extreme Throughput and Optimized Compression Ratio", IEEE/ACM SC2024, **best student paper finalist**.
End-to-end compression: **250-500GB/s**, End-to-end decompression: **300-500GB/s**, in general.
6. cuSZp3: https://github.com/szcompressor/cuSZp (improved compression ratio; shrunk memory footprint)
Yafan Huang, Sheng Di, Guanpeng Li, Franck Cappello, "GPU Lossy Compression for HPC Can Be Versatile and Ultra-Fast", IEEE/ACM The International Conference for High Performance computing, Networking, Storage and Analysis (IEEE/ACM SC2025), 2025.

Handson: Installation of SZ2, SZ3

1. SZ2 (SZ2.1)

(1) Getting SZ: github clone http://github.com/szcompressor/SZ2
Using Spack: spack install sz
Using cmake: mkdir build; cd build; cmake ..; make
Using automake: ./configure –prefix=…; make; make install
(3) Executable command: sz

2. SZ3

(1) Getting SZ3: git clone https://github.com/szcompressor/SZ3 or
wget https://github.com/szcompressor/SZ3/releases/download/v3.3.0/SZ3-v3.3.0.zip & unzip SZ3-v3.3.0.zip
(2) Installation using cmake: mkdir build; cd build; cmake ..; make
(3) Executable command: sz3

Handson: Quick start/test of SZ compression

SZ2's executable command: **sz**
SZ3's executable command: **sz3**

/ # cd /SZ/example

/SZ/example # ./sz -z -f -i testdata/x86/testfloat_8_8_128.dat -3 8 8 128 -M REL -R 0.01

Testfloat_8_8_128.dat's dimension is 128x8x8

```
compression time = 0.001975
compressed data file: testdata/x86/testfloat_8_8_128.dat.sz
```

/SZ/example # ls testdata/x86/ -l

```
-rw-r--r-- 1 root root  65536 May 16 13:05 testdouble_8_8_128.dat
-rw-r--r-- 1 root root 524288 May 16 13:05 testdouble_8_8_8_128.dat
-rw-r--r-- 1 root root  32768 May 16 13:05 testfloat_8_8_128.dat
-rw-r--r-- 1 root root    856 Jun  7 08:32 testfloat_8_8_128.dat.sz
-rw-r--r-- 1 root root  34816 May 16 13:05 testfloat_8_8_128.h5
-rw-r--r-- 1 root root   1024 May 16 13:05 testint16_8x8x8.dat
-rw-r--r-- 1 root root   2048 May 16 13:05 testint32_8x8x8.dat
-rw-r--r-- 1 root root   4096 May 16 13:05 testint64_8x8x8.dat
-rw-r--r-- 1 root root    512 May 16 13:05 testint8_8x8x8.dat
```

Error bound is set in sz.config in this example:
errorBoundMode = REL
relBoundRatio = 1E-2

Handson: Quick start/test of SZ decompression

/SZ/example # ./sz -x -f -s testdata/x86/testfloat_8_8_128.dat.sz -i testdata/x86/testfloat_8_8_128.dat -3 8 8 128 -a

```
Min=0.21328493952751159668, Max=5.053220748901367l875,
range=4.839935779571533203l
Max absolute error = 0.0483615696
Max relative error = 0.009992
Max pw relative error = 0.205610
PSNR = 46.085006, NRMSE= 0.004963061828213806078l
normError = 2.174124, normErr_norm = 0.015142
acEff=0.999827
compressionRatio=38.280374
decompression time = 0.003250 seconds.
decompressed data file: testdata/x86/testfloat_8_8_128.dat.sz.out
```

When executing decompression by the sz command, you can use '**-a**' to show the detailed information about compression and reconstructed data. To this end, you need to use **-i** to specify the original data file.

Supporting python (pysz)

pip install pysz

```
sdi@sdipc:~$ pip install pysz
Defaulting to user installation because normal site-packages is not w
Collecting pysz
  Downloading pysz-1.0.2-cp313-cp313-manylinux_2_24_x86_64.manylinux_
Requirement already satisfied: numpy>=1.20.0 in /usr/lib64/python3.13
Downloading pysz-1.0.2-cp313-cp313-manylinux_2_24_x86_64.manylinux_2_
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 9.4/9.4 MB 1.2 MB/s eta 0
Installing collected packages: pysz
Successfully installed pysz-1.0.2
```

```python
import numpy as np
from pysz import sz, pyConfig

# Create test data
data = np.random.rand(8, 8, 128).astype(np.float32)

# Create config
config = pyConfig(data.shape)
config.errorBoundMode = pyConfig.EB.ABS
config.absErrorBound = 1e-3
config.cmprAlgo = pyConfig.ALGO.INTERP_LORENZO

# Compress
compressed, ratio = sz.compress(data, config)
print(f"Compression ratio: {ratio:.2f}x")

# Decompress
decompressed = sz.decompress(compressed, np.float32, data.shape, config)

# Verify
max_err, psnr, nrmse = sz.verify(data, decompressed)
print(f"Max error: {max_err:.2e}, PSNR: {psnr:.2f} dB, NRMSE: {nrmse:.2e}")
```

https://pypi.org/project/**pysz**/

Pre-built binary wheels are available for most platforms (Linux, macOS, Windows) and Python versions (3.8-3.13)

cuSZp 1~3 (latest GPU version of SZ)

Github: https://github.com/szcompressor/cuSZp

- **cuSZp 3:** https://github.com/szcompressor/cuSZp/releases/download/cuSZp-V3.0.0/cuSZp-V3.0.0.zip

  Yafan Huang, Sheng Di, Guanpeng Li, Franck Cappello, "GPU Lossy Compression for HPC Can Be Versatile and Ultra-Fast", ***IEEE/ACM SC2025***, 2025.

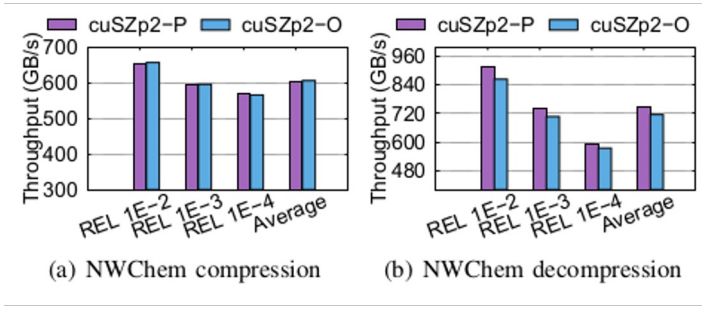
- **cuSZp 2**: https://github.com/szcompressor/cuSZp/releases/download/cuSZp-V2.0.1/cuSZp-V2.0.1.zip

  Yafan Huang, Sheng Di, Guanpeng Li, Franck Cappello, "cuSZp2: A GPU Lossy Compressor with Extreme Throughput and Optimized Compression Ratio", ***IEEE/ACM SC2024***, 2024. **[best student paper finalist]**

- **cuSZp 1**: https://github.com/szcompressor/cuSZp/releases/download/cuSZp-V1.1/cuSZp-V1.1.0.zip

  Yafan Huang, Sheng Di, Xiaodong Yu, Guanpeng Li, Franck Cappello, "cuSZp: An Ultra-fast GPU Error-bounded Lossy Compression Framework with Optimized End-to-End Performance", ***IEEE/ACM SC2023***, 2023

	REL	CESM-ATM	HACC	RTM	SCALE	QMCPack	NYX
cuSZp2-O	1E-2	18.44∼82.41 **(avg: 42.98)**	11.49∼20.09 **(avg: 15.50)**	30.12∼104.18 **(avg: 61.48)**	16.80∼109.55 **(avg: 46.19)**	12.44∼23.57 **(avg: 18.01)**	14.36∼127.80 (avg: 69.14)
	1E-3	12.99∼57.45 **(avg: 24.53)**	5.85∼12.47 **(avg: 8.82)**	12.00∼84.96 **(avg: 40.24)**	11.10∼79.69 **(avg: 29.52)**	6.07∼13.29 **(avg: 9.68)**	10.50∼125.56 (avg: 41.75)
	1E-4	7.85∼39.01 **(avg: 14.97)**	3.67∼6.27 **(avg: 4.84)**	6.51∼67.81 **(avg: 29.36)**	6.31∼49.95 **(avg: 17.92)**	3.79∼7.25 (avg: 5.52)	5.43∼98.37 **(avg: 24.12)**



(a) NWChem compression (b) NWChem decompression

cuSZp 1~3 (latest GPU version of SZ)

Installation using cmake (using cuSZp V3.0 as example)

/ # wget https://github.com/szcompressor/cuSZp/releases/download/cuSZp-V3.0.0/cuSZp-V3.0.0.zip

/ # unzip cuSZp-V3.0.0.zip; cd cuSZp-V3.0.0; mkdir build; cd build

/ build # cmake -DCMAKE_BUILD_TYPE=Release -DCMAKE_INSTALL_PREFIX=../install/ ..

/ build # make -j && make install

Testing cuSZp

/ # cd ./install/bin

/ # ./cuSZp --help

```
cuSZp: A GPU error-bounded lossy compressor for floating-point data

Usage:
  ./cuSZp -i [oriFilePath] -t [dataType] -m [encodingMode] -d [dim] [dim_z] [dim_y] [dim_x] -eb [errorBoundMode] [errorBound] -x [cmpFilePath] -o [decFilePath]

Options:
  -i  [oriFilePath]    Path to the input file containing the original data
  -t  [dataType]       Data type of the input data. Options:
                       f32      : Single precision (float)
                       f64      : Double precision (double)
  -m  [encodingMode]   Encoding mode to use. Options:
                       fixed    : No-delta fixed-length encoding mode
                       plain    : Plain fixed-length encoding mode (with delta)
                       outlier  : Outlier fixed-length encoding mode (with delta and outlier preservation)
  -d  [dim]            Data dimensionality and processing manner. Options:
                       1                          : 1D Processing Manner (can be used for all datasets)
                       2 [dim_z] [dim_y] [dim_z] : 2D Processing Manner (can be used for both 2D and 3D datasets)
                       3 [dim_z] [dim_y] [dim_z] : 3D Processing Manner (can be used for only 3D datasets)
                       Note: dim_x is the fastest varying dimension.
                       Hint: 2D processing can be used for 3D datasets by compressing each [dim_y x dim_x] slice independently.
  -eb [errorBoundMode] [errorBound]
                       errorBoundMode can only be:
                       abs      : Absolute error bound
                       rel      : Relative error bound
                       errorBound is a floating-point number representing the error bound, e.g. 1E-4, 0.03.
  -x  [cmpFilePath]    Path to the compressed output file (optional)
  -o  [decFilePath]    Path to the decompressed output file (optional)
```

cuSZp 1~3 (latest GPU version of SZ)

Two Examples (on my laptop with NVIDIA Quadro T2000 (4GB GDDR6 VRAM))

1. Running cuSZp using the **P** mode based on relative error bound 1E-3 on Uf01.dat

```
[sdi@sdihost bin]$ ./cuSZp -i ~/Data/Hurricane-ISA/Uf01.dat -t f32 -m plain -eb rel 1E-3
cuSZp finished!
cuSZp compression   end-to-end speed: 27.609826 GB/s
cuSZp decompression end-to-end speed: 68.194415 GB/s
cuSZp compression ratio: 5.042417

Pass error check!
```

1. Running cuSZp using the **O** mode based on relative error bound 1E-3 on Uf01.dat

```
[sdi@sdihost bin]$ ./cuSZp -i ~/Data/Hurricane-ISA/Uf01.dat -t f32 -m outlier -eb rel 1E-3
cuSZp finished!
cuSZp compression   end-to-end speed: 25.200892 GB/s
cuSZp decompression end-to-end speed: 54.260033 GB/s
cuSZp compression ratio: 11.310083

Pass error check!
```

Supporting python (pycuSZp)

ctypes, numpy, and pycuda are required packages for pycuSZp (can be installed with pip, e.g., *pip install pycuda*).

Install cuSZp, then go to ./python/ folder. There are two examples of using pycuSZp.

/ # wget https://github.com/szcompressor/cuSZp/releases/download/cuSZp-V3.0.0/cuSZp-V3.0.0.zip

/ # unzip cuSZp-V3.0.0.zip; cd cuSZp-V3.0.0; mkdir build; cd build

/ build # cmake -DCMAKE_BUILD_TYPE=Release -DCMAKE_INSTALL_PREFIX=../install/ ..

/ build # make -j && make install && cd ../python/

pycuSZp.py

yafan > 25-SC-cuSZp3 > 24-22-cuSZp3-code-ready > cuSZp > python > pycuSZp.py > cuSZp > compress

```python
import ctypes
import numpy as np

# Load the shared library (using default installation path, replace if neccessary)
lib = ctypes.CDLL('../install/lib/libcuSZp.so')

# Define uint3 in ctypes
class Uint3(ctypes.Structure):
    _fields_ = [("x", ctypes.c_uint), ("y", ctypes.c_uint), ("z", ctypes.c_uint)]

# Update function signatures with dim and dims
lib.cuSZp_compress.argtypes = [
    ctypes.c_void_p,                    # d_oriData
    ctypes.c_void_p,                    # d_cmpBytes
    ctypes.c_size_t,                    # nbEle
    ctypes.POINTER(ctypes.c_size_t),    # cmpSize
    ctypes.c_float,                     # errorBound
    ctypes.c_int,                       # dim (cuszp_dim_t),   1 for 1D, 2 for 2D, 3 for 3D
    Uint3,                              # dims (uint3)
    ctypes.c_int,                       # type (cuszp_type_t), 0 for f32, 1 for f64
    ctypes.c_int,                       # mode (cuszp_mode_t), 0 for fixed, 1 for plain, 2 for outlier
    ctypes.c_void_p                     # stream
]
```

Path to cuSZp dynamic library (libcuSZp.so).

Running pycuszp to compress a 3D data (tested on an A100 GPU)
Executing "python example-hpc.py" can check the detailed usage.
python example-hpc.py ./pressure_3000 f32 outlier 3 1008 1008 352 1e-4

```
===== cuSZp GPU Compression Test =====
Data file:          /home/yafan.huang/yafan/snapshots-pressure/pressure_3000
Type:               f32
Mode:               outlier
Dim:                3D (1008, 1008, 352)
Error bound:        0.0001
Original size:      1364.34 MB
Compressed size:    378.39 MB
Compression ratio: 3.61x
Compression speed: 73.41 GB/s
Decompression speed: 153.31 GB/s
Within bound: True
```

Summary

SZ 1.4: https://github.com/szcompressor/SZ2
(withLinearRegression = NO in sz.config) [IPDPS17]
SZ 2.0: https://github.com/szcompressor/SZ2
(withLinearRegression = YES in sz.config) [BigData 20]
SZ 3.0-3.2:
https://github.com/szcompressor/SZ3/releases/tag/v3.2.1 [ICDE21]
SZ 3.3+: https://github.com/szcompressor/SZ3/releases/tag/v3.3.0
[SIGMOD24]
cuSZp (CUDA): https://github.com/szcompressor/cuSZp
[SC23, SC24, SC25]
SZp (CPU): https://github.com/szcompressor/SZp [SC24]
SZx (CPU & CUDA): https://github.com/szcompressor/SZx [HPDC22]
QCAT: https://github.com/szcompressor/QCAT

Acknowledgment:
This research was supported by the Exascale Computing Project (ECP), Project Number: 17-SC-20-SC, a collaborative effort of two DOE organizations – the Office of Science and the National Nuclear Security Administration, responsible for the planning and preparation of a capable exascale ecosystem, including software, applications, hardware, advanced system engineering and early testbed platforms, to support the nation's exascale computing imperative. The material was supported by the U.S. Department of Energy, Office of Science, under contract DE-AC02-06CH11357, and NSF Grant OAC-2104023, OAC-2311875, OAC-2514036, OAC-2513768.

Acknowledgment

This research was supported by the Exascale Computing Project (ECP), Project Number: 17-SC-20-SC, a collaborative effort of two DOE organizations – the Office of Science and the National Nuclear Security Administration, responsible for the planning and preparation of a capable exascale ecosystem, including software, applications, hardware, advanced system engineering and early testbed platforms, to support the nation's exascale computing imperative. The material was supported by the U.S. Department of Energy, Office of Science, under contract DE-AC02-06CH11357, and NSF Grant OAC-2104023, OAC-2311875, OAC-2514036, OAC-2513768.